Depression-Related Facial Expression Classification

A CNN Trained from Scratch versus Transfer Learning with MobileNetV2 and ResNet50

Artificial Neural Networks and Deep Learning — Final Project (Topic 2)

October 2026

Team members

Sara Bedulla — data pipeline, exploratory analysis, CNN from scratch

Kejdi Shahu — transfer learning, evaluation and results

Code: https://github.com/SaraBedulla/FacemoodClassifier

Google Colab: FacemoodClassifier_colab.ipynb

Abstract

Depression is associated with more negative and less positive facial affect. This project classifies facial-expression images into the seven basic emotions and into three depression-related expression patterns (negative, neutral, positive), and compares a convolutional neural network (CNN) trained from scratch with two ImageNet-pretrained transfer-learning models, MobileNetV2 and ResNet50. All models use the same fixed data splits, augmentation, class weighting and evaluation on two public datasets: FER-2013 (35,887 grayscale 48×48 images) and RAF-DB (15,339 colour 100×100 images). The scratch CNN achieved the best macro-F1 in three of four settings: on FER-2013 it reached 64.8% test accuracy (macro-F1 0.633) for seven emotions, and on RAF-DB 78.7% (0.700) for seven emotions and 82.4% (0.808) for the three affect classes. ResNet50 was best only on FER-2013 affect classes (76.3%, macro-F1 0.740), by less than two points. Fine-tuning was essential for the pretrained models: a frozen ImageNet backbone reached only 45–67% validation accuracy, and fine-tuning the top stage added 10–19 percentage points. The scratch CNN was also the fastest model at inference (2–13 times faster than the pretrained models). With small, low-resolution face images and conservative fine-tuning, ImageNet pretraining did not give an advantage over a well-regularised CNN trained directly on faces.

Contents

1. Introduction and Problem Statement 4
2. Dataset Description 4
2.1 FER-2013 4
2.2 RAF-DB 5
2.3 Label Schemes 7
2.4 Data Splits and Preprocessing 7
3. Methodology and Model Architecture 8
3.1 Pipeline Overview 8
3.2 Input Processing, Augmentation and Class Imbalance 8
3.3 CNN Trained from Scratch 8
3.4 Transfer-Learning Models 9
3.5 Two-Phase Fine-Tuning 9
3.6 Training Configuration 9
3.7 Evaluation Metrics 10
4. Experimental Results 10
4.1 Main Comparison 10
4.2 Training and Validation Curves 11
4.3 Effect of Fine-Tuning 14
4.4 Per-Class Performance 15
4.5 Confusion Matrices 15
4.6 Sample Predictions 17
4.7 Efficiency 18
5. Output Screenshots from Google Colab 18
6. Analysis and Discussion 19
6.1 Did Transfer Learning Help? 19
6.2 Why Did the Pretrained Models Not Win? 19
6.3 Accuracy versus Efficiency 20
6.4 Emotions versus Depression-Related Patterns 20
6.5 Comparison with Published Results 20
6.6 Ethical Considerations 20
7. Limitations and Future Work 20
7.1 Limitations 20
7.2 Future Work 21
8. Conclusion 21
References 21
Appendix A. Reproducing the Results 23
Appendix B. Training Curves and Confusion Matrices of the Remaining Models 24

1. Introduction and Problem Statement

Depression is one of the most common mental disorders worldwide. Clinical and behavioural studies have found that people with depression tend to show fewer positive facial expressions, more negative or flat affect, and reduced expressiveness overall (Cohn et al., 2009; Girard et al., 2014). Automatic facial expression recognition (FER) could therefore support research on depression-related behaviour, for example by summarising how often a person shows negative, neutral or positive expressions over time.

Deep CNNs are the standard approach to FER (Li & Deng, 2022). Two strategies are common. A network can be **trained from scratch** on face images, so all of its features are learned for faces, or a network pretrained on a large general dataset such as ImageNet (Deng et al., 2009) can be reused through **transfer learning**, keeping its generic visual features and adapting them to the new task (Yosinski et al., 2014). Transfer learning is usually expected to help when the target dataset is small, but face images differ strongly from the object photographs in ImageNet, so the benefit for facial expressions is not obvious.

Problem statement. Given a single face image, predict (a) one of the seven basic emotions — angry, disgust, fear, happy, sad, surprise, neutral — and (b) one of three depression-related expression patterns: negative, neutral or positive affect.

Research question. How does a CNN trained from scratch compare with transfer learning (MobileNetV2 and ResNet50) for facial expression classification, in terms of accuracy and efficiency?

The main contributions of the project are:

- A reproducible pipeline (data preparation, training, evaluation and comparison) that runs on Google Colab and locally, with fixed data splits shared by every model.
- A controlled comparison of three models on two datasets and two label schemes (12 trained models), using accuracy, macro-F1, per-class recall, confusion matrices, model size, training time and inference time.
- An analysis of why the pretrained models did not outperform the scratch CNN, including the effect of fine-tuning the pretrained backbone.

2. Dataset Description

Two public facial-expression datasets were used. Both have the same seven emotion classes, which makes results comparable across datasets. Neither dataset contains clinical or depression labels; the depression-related classes are derived from the emotion labels (Section 2.3).

2.1 FER-2013

FER-2013 was introduced in the ICML 2013 *Challenges in Representation Learning* competition (Goodfellow et al., 2013). The images were collected automatically with Google image search, cropped to the face and labelled by a single annotator, so some labels are wrong; human accuracy on the dataset is about 65%. It contains 35,887 grayscale images of 48×48 pixels.

Link: https://www.kaggle.com/datasets/msambare/fer2013

2.2 RAF-DB

The Real-world Affective Faces Database (RAF-DB; Li et al., 2017) contains 15,339 aligned colour face images of 100×100 pixels collected from the internet. Each image was labelled by about 40 annotators, so its labels are much more reliable than those of FER-2013. The official version is available for non-commercial research on request; a Kaggle copy with the same images was used.

Links: https://www.kaggle.com/datasets/shuvoalok/raf-db-dataset (official: http://www.whdeng.cn/RAF/model1.html)

Table 1. *Overview of the two datasets.*

	FER-2013	**RAF-DB**
Images	35,887	15,339
Format	48×48 grayscale	100×100 colour, aligned
Labels	1 annotator per image (noisy)	~40 annotators per image
Train / validation / test	25,838 / 2,871 / 7,178	11,043 / 1,228 / 3,068
Largest class	happy (25.0%)	happy (38.8%)
Smallest class	disgust (1.5%)	fear (2.3%)

Table 2. *Number of images per emotion (all splits) and share of the dataset.*

Emotion	**FER-2013**	**Share**	**RAF-DB**	**Share**
angry	4,953	13.8%	867	5.7%
disgust	547	1.5%	877	5.7%
fear	5,121	14.3%	355	2.3%
happy	8,989	25.0%	5,957	38.8%
sad	6,077	16.9%	2,460	16.0%
surprise	4,002	11.2%	1,619	10.6%
neutral	6,198	17.3%	3,204	20.9%
Total	35,887		15,339	

Both datasets are strongly imbalanced. In FER-2013 *happy* has 16.6 times more training images than *disgust*; in RAF-DB *happy* has 17 times more than *fear*. Figures 1 and 2 show the class distributions per split, and Figure 3 shows random training samples.

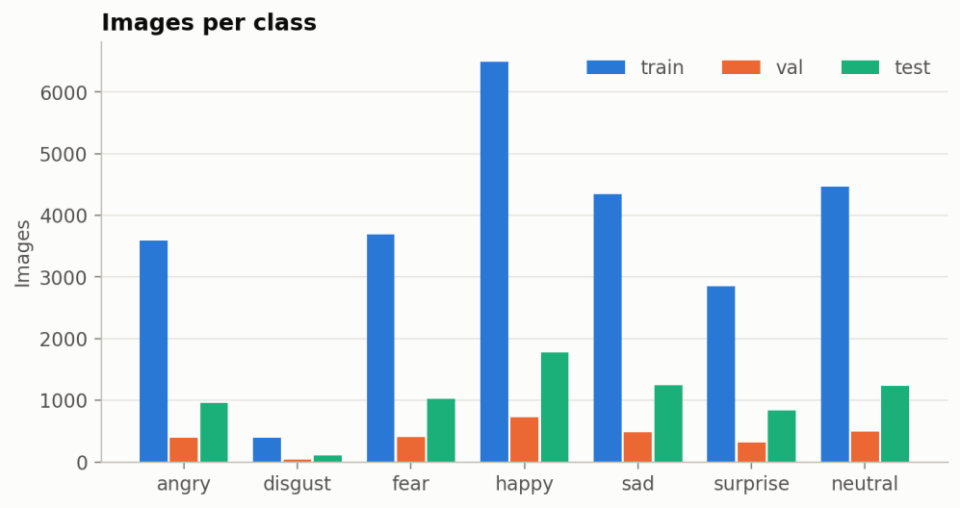


Figure 1. *FER-2013: images per emotion in the train, validation and test splits.*

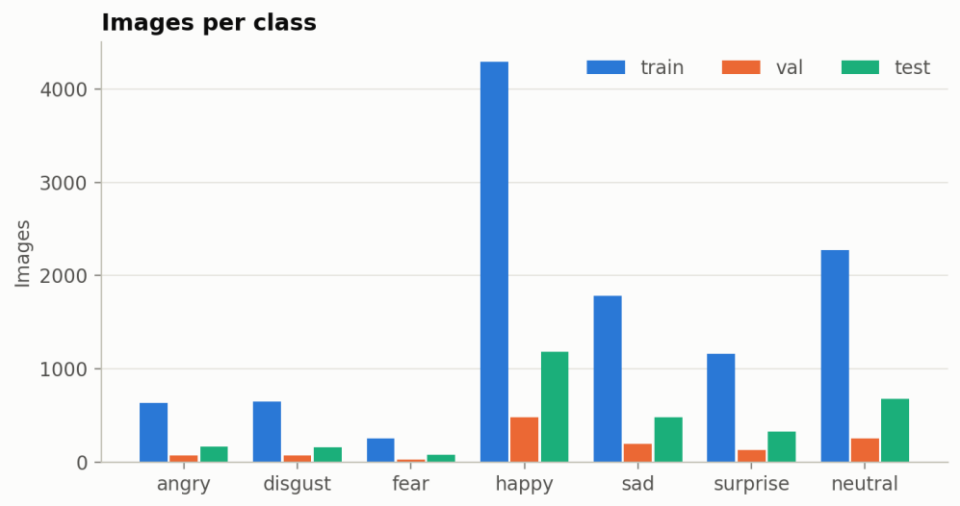


Figure 2. *RAF-DB: images per emotion in the train, validation and test splits.*



Figure 3. *Random FER-2013 training images, one row per emotion.*

2.3 Label Schemes

Two label schemes are used. **emotion7** keeps the seven emotions. **affect3** groups them into three depression-related expression patterns, following findings that depression is associated with more negative and less positive facial affect: *negative* (angry, disgust, fear, sad), *neutral* (neutral) and *positive* (happy, surprise).

Table 3. *affect3 classes and their share of each dataset.*

Class	Emotions	FER-2013	RAF-DB
negative	angry, disgust, fear, sad	46.5%	29.7%
neutral	neutral	17.3%	20.9%
positive	happy, surprise	36.2%	49.4%

2.4 Data Splits and Preprocessing

For both datasets the official test set is kept as the test set, and a stratified 10% of the official training set is held out for validation with a fixed random seed (42). The splits are built once and saved, so every model is trained, tuned and tested on exactly the same images. Images are stored in their native format (FER-2013 48×48×1, RAF-DB 100×100×3) and converted on the fly to the input each model needs (Section 3). Figure 4 shows the mean face of each emotion, which already reveals typical shapes such as the open mouth for *surprise* and the smile for *happy*.



Figure 4. *FER-2013: mean training image of each emotion.*

3. Methodology and Model Architecture

3.1 Pipeline Overview

The project is organised as a small Python package. `src/data/prepare.py` builds the fixed splits; `src/data/dataset.py` loads them and creates `tf.data` input pipelines with augmentation; `src/models/` contains the three models; `src/training/train.py` trains one configuration; `src/evaluation/evaluate.py` computes the test-set metrics and figures for every trained model; and `src/evaluation/compare.py` combines all runs into comparison tables and charts. Each training run writes its configuration, training history, curves, best model and test results to its own folder, so every number in this report can be traced back to a run. The Google Colab notebook calls the same commands.

3.2 Input Processing, Augmentation and Class Imbalance

Pixels are kept as raw 0–255 values in the input pipeline and every model applies its own normalisation inside the network, so all models receive the same data. During training only, images are augmented with random horizontal flips, rotations (up to ±18°), zoom (±10%) and shifts (±10% horizontally and vertically). To counter class imbalance, the loss is weighted with *balanced* class weights, inversely proportional to class frequency, so mistakes on rare classes such as *disgust* and *fear* cost more.

3.3 CNN Trained from Scratch

The baseline is a VGG-style CNN with four convolutional blocks (Table 4). Each block applies two 3×3 convolutions, each followed by batch normalisation (Ioffe & Szegedy, 2015) and ReLU, then 2×2 max-pooling and dropout (Srivastava et al., 2014). Global average pooling replaces flattening, which keeps the classifier small and lets the same network work at any input size. The network takes each dataset at its native resolution, rescaled to [0, 1] inside the model, and has about 4.8 million parameters.

Table 4. *Architecture of the scratch CNN (output shapes for FER-2013 48×48 input; RAF-DB 100×100 gives 50, 25, 12 and 6 pixels after each block).*

Stage	Layers	Output shape
Input	Rescaling to [0, 1]	48×48×1
Block 1	2 × [Conv 3×3, 64 filters → BatchNorm → ReLU], MaxPool 2×2, Dropout 0.25	24×24×64
Block 2	2 × [Conv 3×3, 128 filters → BatchNorm → ReLU], MaxPool 2×2, Dropout 0.25	12×12×128
Block 3	2 × [Conv 3×3, 256 filters → BatchNorm → ReLU], MaxPool 2×2, Dropout 0.25	6×6×256
Block 4	2 × [Conv 3×3, 512 filters → BatchNorm → ReLU], MaxPool 2×2, Dropout 0.25	3×3×512

Stage	Layers	Output shape
Head	Global average pooling → Dense 256 → BatchNorm → ReLU → Dropout 0.5	256
Output	Dense with softmax	7 (or 3)

3.4 Transfer-Learning Models

Two ImageNet-pretrained backbones from Keras Applications were used, chosen to represent the two ends of the size–accuracy trade-off. **MobileNetV2** (Sandler et al., 2018) is a lightweight network built from inverted residual blocks with depthwise separable convolutions, designed for mobile devices. **ResNet50** (He et al., 2016) is a 50-layer residual network with skip connections and about ten times more parameters.

Both backbones are used without their ImageNet classifier and receive a new head: global average pooling, dropout 0.3 and a softmax layer. Faces are resized to 224×224 pixels, the resolution the backbones were pretrained at, and grayscale FER-2013 images are copied to three channels. Each backbone's own ImageNet preprocessing is part of the model (MobileNetV2 scales pixels to [-1, 1]; ResNet50 converts RGB to BGR and subtracts the ImageNet channel means).

Table 5. *Transfer-learning models (7-class versions).*

	MobileNetV2	ResNet50
Input	224×224×3	224×224×3
Backbone output	7×7×1280	7×7×2048
New head	GAP → Dropout 0.3 → Dense softmax	GAP → Dropout 0.3 → Dense softmax
Total parameters	2.27M	23.6M
Trainable in phase 1 (head only)	8,967	14,343
Layers unfrozen in phase 2	last 38 of 154 (blocks 13–16)	last 32 of 175 (stage conv5)
Trainable in phase 2	1.67M	15.0M

3.5 Two-Phase Fine-Tuning

The pretrained models are trained in two phases. In **phase 1** the backbone is frozen and only the new head is trained for up to 10 epochs with learning rate 10^{-3}. This prevents the large, random gradients of the untrained head from destroying the pretrained features. In **phase 2** the top stage of the backbone is unfrozen and trained together with the head at a learning rate 100 times smaller (10^{-5}), so the high-level features adapt to faces while the generic low-level features stay intact. Batch-normalisation layers stay frozen and run in inference mode throughout, so the small face datasets do not overwrite the ImageNet statistics. The model with the lowest validation loss over both phases is kept.

3.6 Training Configuration

All models are trained with the Adam optimiser (Kingma & Ba, 2015) and sparse categorical cross-entropy weighted by the class weights. Early stopping monitors validation loss and restores the best weights; the learning rate is halved when the validation loss stops improving. Table 6 lists the settings. They are identical for all models, except for one run: the RAF-DB emotion7 scratch CNN, which was

trained with a smaller learning rate, a longer budget and more patience after the default settings stopped too early.

Table 6. *Training settings.*

Setting	Value
Optimiser	Adam
Batch size	64
Learning rate	10^{-3} (scratch CNN and phase 1); 10^{-5} (phase 2 fine-tuning)
Maximum epochs	60 in total (phase 1: up to 10)
Early stopping	Validation loss, patience 10 epochs, best weights restored
Learning-rate schedule	×0.5 after 5 epochs without improvement, minimum 10^{-6}
Loss	Sparse categorical cross-entropy with balanced class weights
Random seed	42 (data split, initialisation, augmentation)
Exception	RAF-DB emotion7 scratch CNN: learning rate $3 \cdot 10^{-4}$, up to 100 epochs, patience 20
Hardware	Google Colab GPU runtime; TensorFlow 2 / Keras 3

3.7 Evaluation Metrics

Each trained model is evaluated once on the held-out test set. Because the classes are imbalanced, accuracy alone can hide poor performance on rare classes, so **macro-F1** (the unweighted mean of the per-class F1 scores) is the main metric. The evaluation also reports per-class recall, the confusion matrix (row-normalised), sample correct and incorrect predictions, the number of parameters, the training time and the average inference time per image on the GPU.

4. Experimental Results

4.1 Main Comparison

Table 7 lists the test results of all 12 models, and Figure 5 compares their macro-F1. The best accuracy and macro-F1 in each setting are in bold.

Table 7. *Test-set results of every model (seed 42). Training time includes early stopping, so it depends on the number of epochs trained.*

Dataset	Labels	Model	Accuracy	Macro-F1	Params	Epochs	Train (min)	ms / image
FER-2013	emotion7	Scratch CNN	64.8%	0.633	4.8M	60	27.4	0.17
FER-2013	emotion7	MobileNetV2	63.0%	0.605	2.3M	60	38.4	1.15
FER-2013	emotion7	ResNet50	63.8%	0.603	23.6M	34	44.5	2.33
FER-2013	affect3	Scratch CNN	74.5%	0.723	4.8M	33	14.9	0.18
FER-2013	affect3	MobileNetV2	76.0%	0.733	2.3M	44	27.8	1.14
FER-2013	affect3	ResNet50	76.3%	0.740	23.6M	29	37.8	2.34
RAF-DB	emotion7	Scratch CNN	78.7%	0.700	4.8M	98	73.7	0.64
RAF-DB	emotion7	MobileNetV2	65.4%	0.575	2.3M	53	14.4	1.31
RAF-DB	emotion7	ResNet50	69.7%	0.608	23.6M	31	17.4	2.48
RAF-DB	affect3	Scratch CNN	82.4%	0.808	4.8M	47	31.9	0.65

Dataset	Labels	Model	Accuracy	Macro-F1	Params	Epochs	Train (min)	ms / image
RAF-DB	affect3	MobileNetV2	75.6%	0.740	2.3M	57	15.5	1.27
RAF-DB	affect3	ResNet50	78.1%	0.762	23.6M	33	18.7	2.48

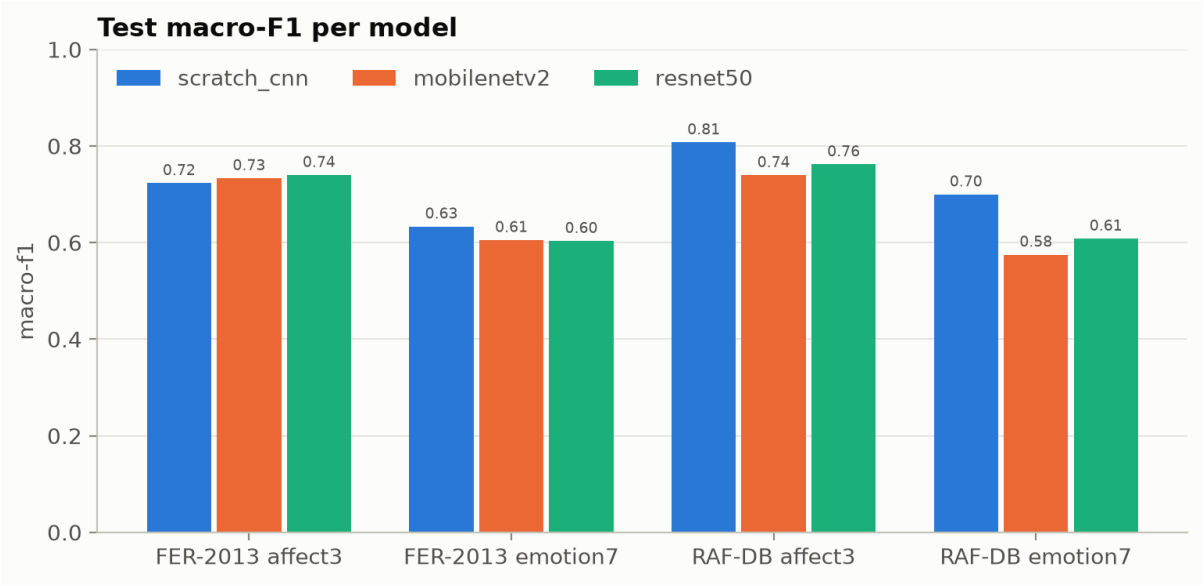


Figure 5. *Test macro-F1 of each model in the four settings.*

The scratch CNN achieved the best macro-F1 in three of the four settings. On **FER-2013** the three models are close: for seven emotions the scratch CNN is about three points ahead in macro-F1 (0.633 vs 0.605 and 0.603), while for the three affect classes ResNet50 is best (76.3% accuracy, macro-F1 0.740), only about one point ahead of the other two. On **RAF-DB** the gap is larger: the scratch CNN reaches 78.7% accuracy (macro-F1 0.700) for seven emotions and 82.4% (0.808) for the affect classes, 4–13 accuracy points above the pretrained models. Among the pretrained models, ResNet50 was better than MobileNetV2 in three of four settings.

All models perform better on RAF-DB than on FER-2013, and on the three affect classes than on the seven emotions. RAF-DB has cleaner labels and higher-resolution colour images, and the affect classes merge emotions that are often confused with each other (for example *fear* and *sad*).

4.2 Training and Validation Curves

Figures 6–11 show the training and validation loss and accuracy of the seven-emotion models on both datasets; the curves of the remaining models are in Appendix B. For the pretrained models the dashed line marks the start of fine-tuning.

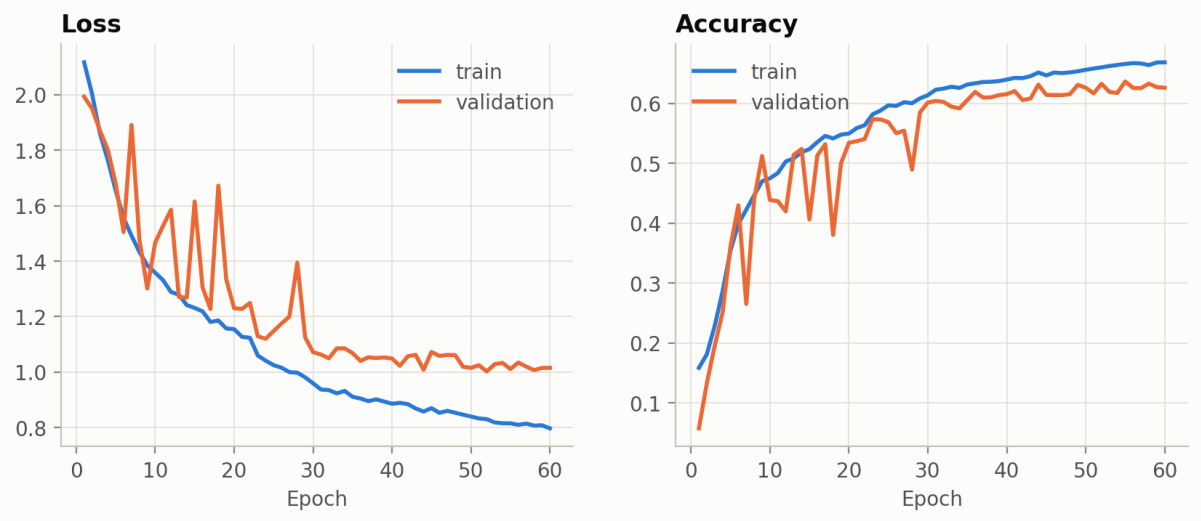


Figure 6. *FER-2013, emotion7, scratch CNN: training and validation curves.*

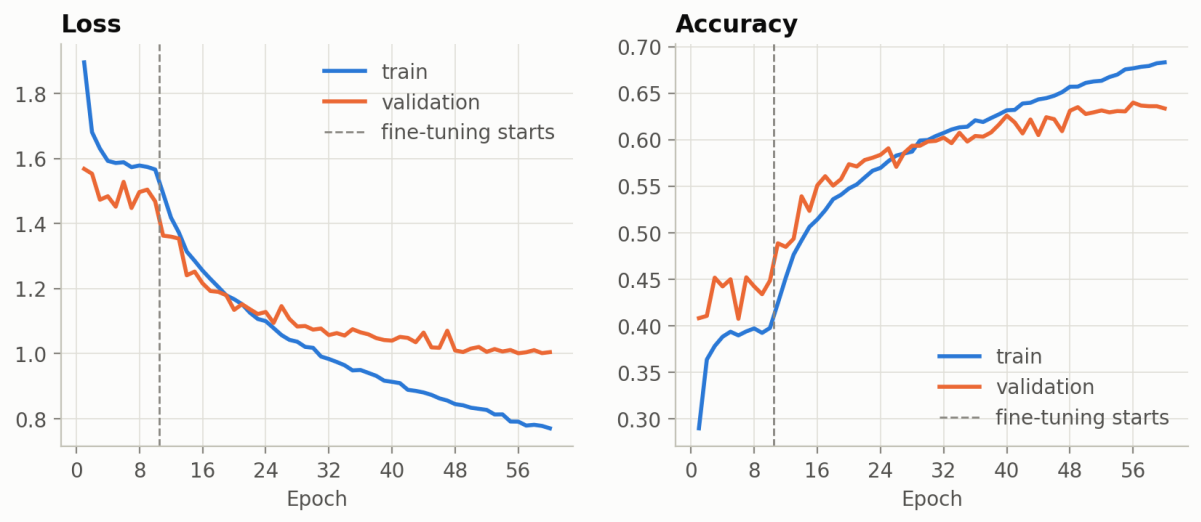


Figure 7. *FER-2013, emotion7, MobileNetV2 (dashed line: start of fine-tuning).*

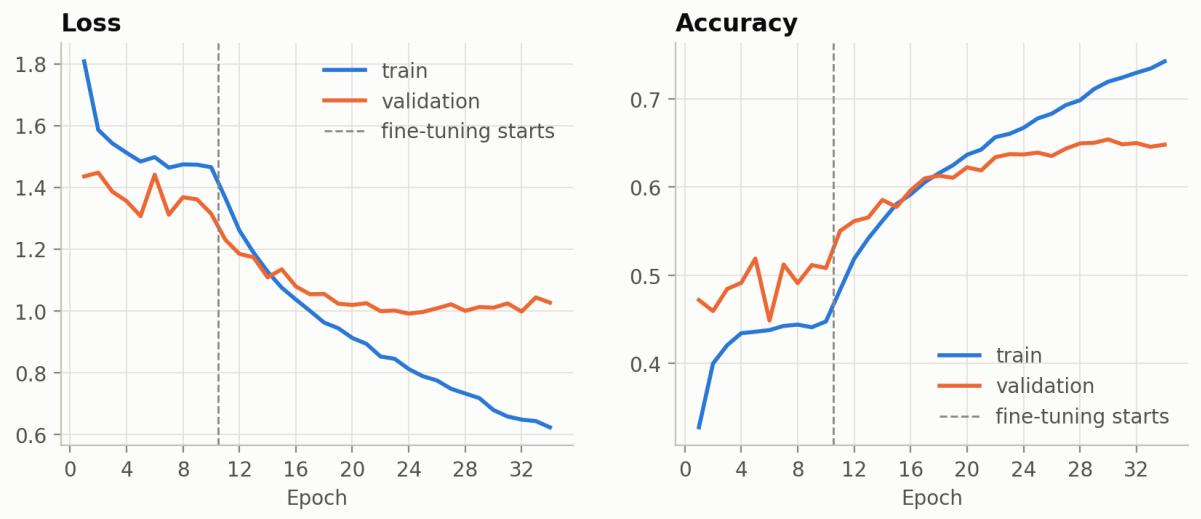


Figure 8. *FER-2013, emotion7, ResNet50 (dashed line: start of fine-tuning).*

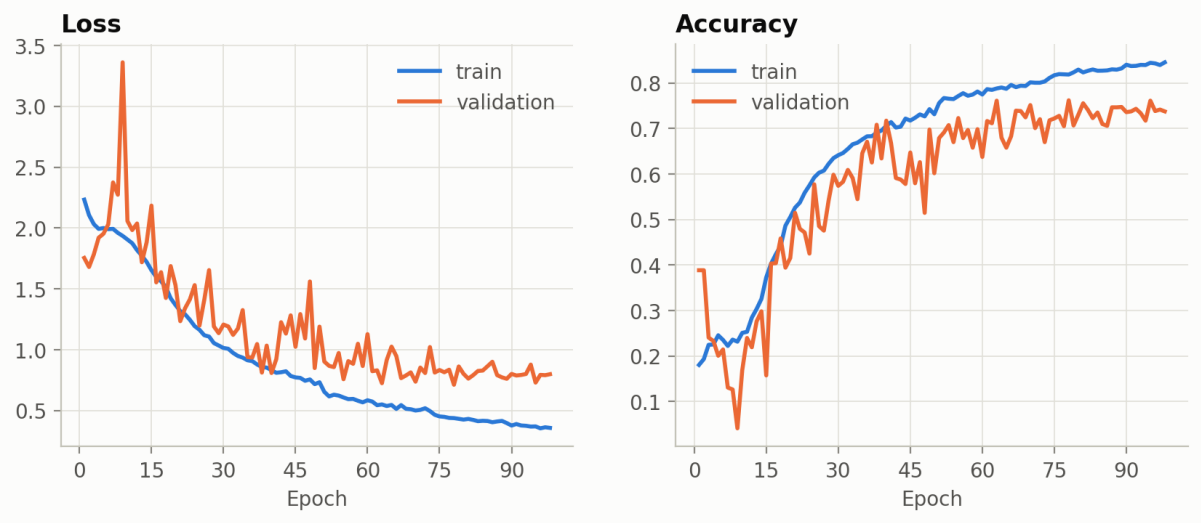


Figure 9. *RAF-DB, emotion7, scratch CNN.*

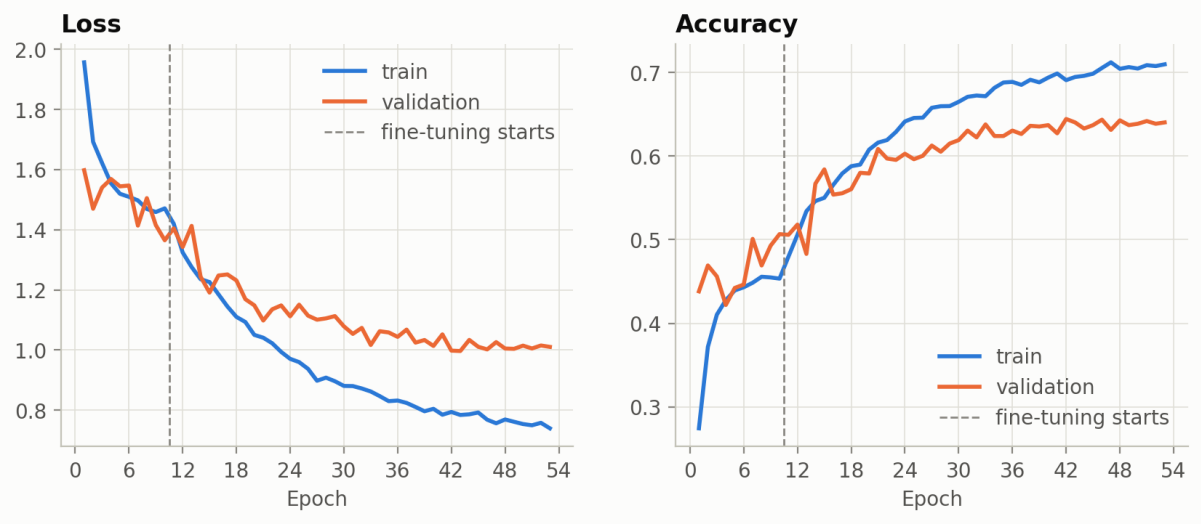


Figure 10. *RAF-DB, emotion7, MobileNetV2.*

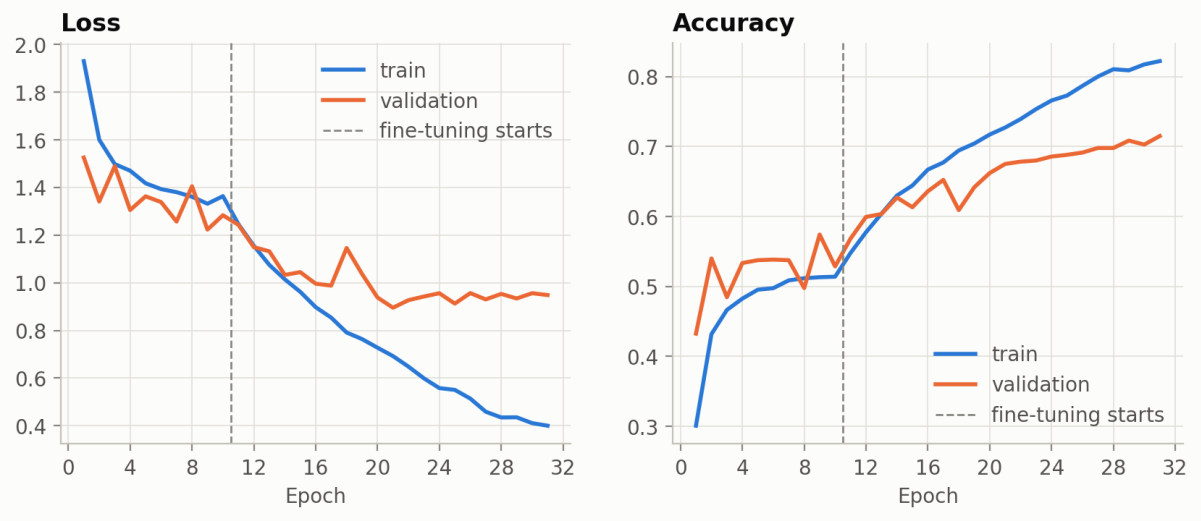


Figure 11. *RAF-DB, emotion7, ResNet50.*

Three patterns are visible. First, the scratch CNN's training and validation curves stay closer together than those of the pretrained models, so dropout, batch normalisation and augmentation kept

overfitting moderate. Second, the pretrained models improve slowly while the backbone is frozen and then jump as soon as fine-tuning starts. Third, after fine-tuning ResNet50 overfits quickly: its training accuracy keeps rising (to 0.74–0.92) while validation accuracy flattens, and early stopping ends training after about 30 epochs. MobileNetV2 overfits less but learns slowly; on FER-2013 emotion7 its best epoch was 56 of 60, so it was still improving when the epoch limit was reached.

4.3 Effect of Fine-Tuning

Table 8 and Figure 12 compare the best validation accuracy with the frozen ImageNet backbone (phase 1) against the best validation accuracy after fine-tuning (phase 2).

Table 8. *Validation accuracy of the pretrained models before and after fine-tuning (pp = percentage points).*

Model	Setting	Frozen backbone	After fine-tuning	Gain
MobileNetV2	FER-2013 affect3	63.2%	76.0%	+12.8 pp
ResNet50	FER-2013 affect3	66.6%	76.5%	+9.9 pp
MobileNetV2	FER-2013 emotion7	45.2%	64.0%	+18.8 pp
ResNet50	FER-2013 emotion7	51.9%	63.7%	+11.8 pp
MobileNetV2	RAF-DB affect3	61.8%	75.7%	+13.9 pp
ResNet50	RAF-DB affect3	67.3%	78.1%	+10.8 pp
MobileNetV2	RAF-DB emotion7	50.7%	64.0%	+13.4 pp
ResNet50	RAF-DB emotion7	57.4%	67.5%	+10.1 pp

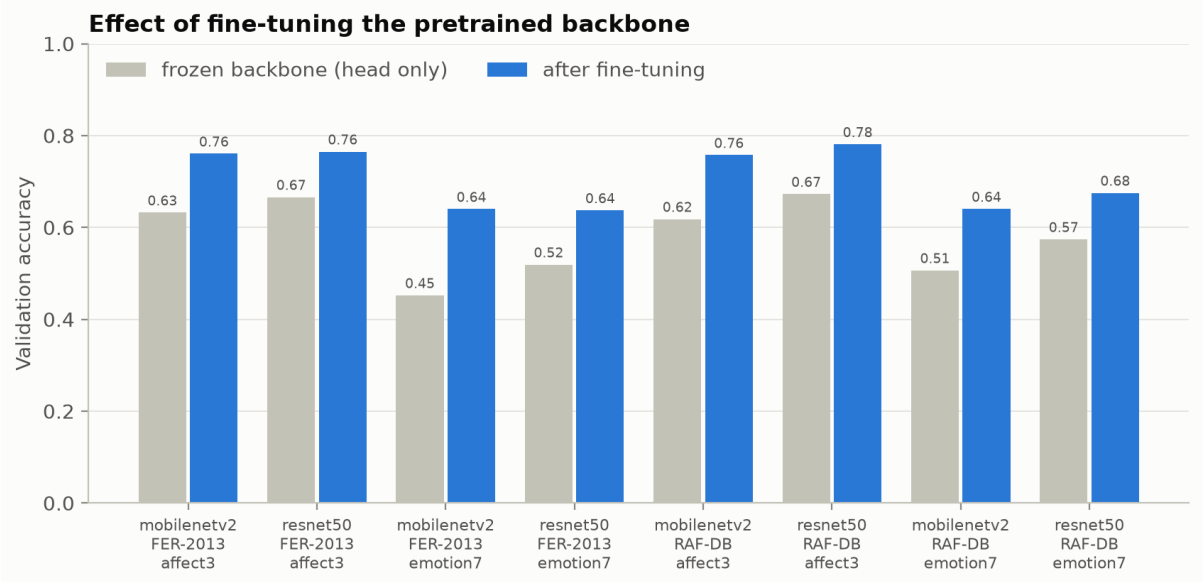


Figure 12. *Validation accuracy with the frozen backbone (head only) and after fine-tuning.*

Frozen ImageNet features reached only 45–67% validation accuracy, well below the scratch CNN, and fine-tuning added 10–19 percentage points in every setting. ResNet50's frozen features were consistently better than MobileNetV2's (by 3–7 points), but after fine-tuning the two were within 0–4 points of each other. This shows that the generic ImageNet features are not directly suited to facial expressions and must be adapted substantially.

4.4 Per-Class Performance

Table 9 shows the recall of each emotion, i.e. the share of test images of that emotion that were classified correctly.

Table 9. *Per-class test recall, emotion7.*

Class	FER Scratch	FER MobileNetV2	FER ResNet50	RAF Scratch	RAF MobileNetV2	RAF ResNet50
angry	0.578	0.590	0.560	0.728	0.661	0.531
disgust	0.676	0.712	0.649	0.450	0.488	0.500
fear	0.424	0.377	0.323	0.649	0.568	0.500
happy	0.808	0.855	0.844	0.852	0.694	0.794
sad	0.488	0.433	0.508	0.776	0.649	0.659
surprise	0.842	0.755	0.773	0.806	0.657	0.799
neutral	0.684	0.657	0.705	0.779	0.634	0.610

Happy and *surprise* are the easiest emotions for every model. On FER-2013, *fear* is the hardest (recall 0.32–0.42), followed by *sad*; both are often confused with each other and with *neutral*. On RAF-DB the rare *disgust* class is the hardest (0.45–0.50) even with class weighting. The pretrained models reached a surprisingly low *happy* recall on RAF-DB (0.69–0.79 vs 0.85 for the scratch CNN): class weighting lowers the weight of this very frequent class, and the pretrained models appear to have been more affected by this than the scratch CNN, which was trained with the same weights.

Table 10. *Per-class test recall, affect3 (depression-related patterns).*

Class	FER Scratch	FER MobileNetV2	FER ResNet50	RAF Scratch	RAF MobileNetV2	RAF ResNet50
negative	0.725	0.737	0.711	0.730	0.746	0.778
neutral	0.704	0.672	0.720	0.884	0.750	0.715
positive	0.790	0.831	0.850	0.851	0.765	0.813

For the depression-related task the most relevant class is *negative*. Its recall ranges from 0.71 to 0.78; the pretrained models recognise negative expressions about as well as the scratch CNN (and ResNet50 slightly better on RAF-DB), but the scratch CNN is much better at separating *neutral* faces on RAF-DB (0.88 vs 0.72–0.75).

4.5 Confusion Matrices

Figures 13–16 show row-normalised confusion matrices: each row shows how the test images of one true class were predicted. The confusion matrices of all other models are in Appendix B.

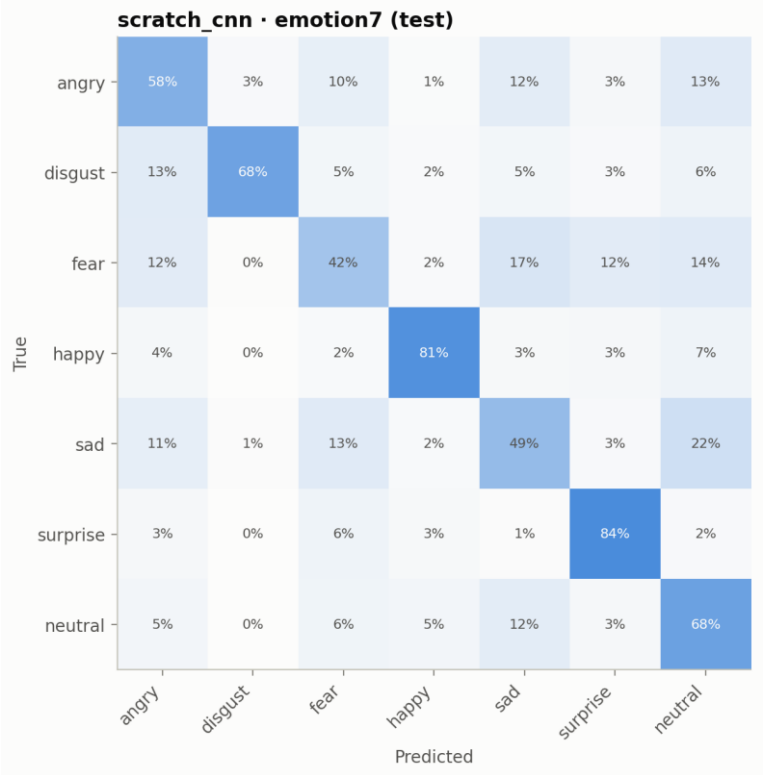


Figure 13. *Confusion matrix, FER-2013, emotion7, scratch CNN.*

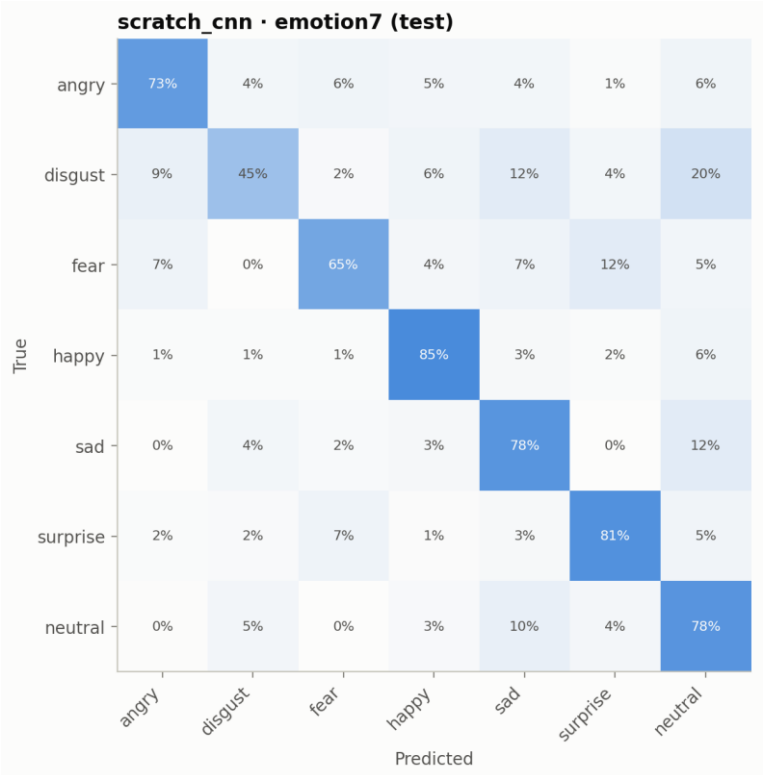


Figure 14. *Confusion matrix, RAF-DB, emotion7, scratch CNN.*

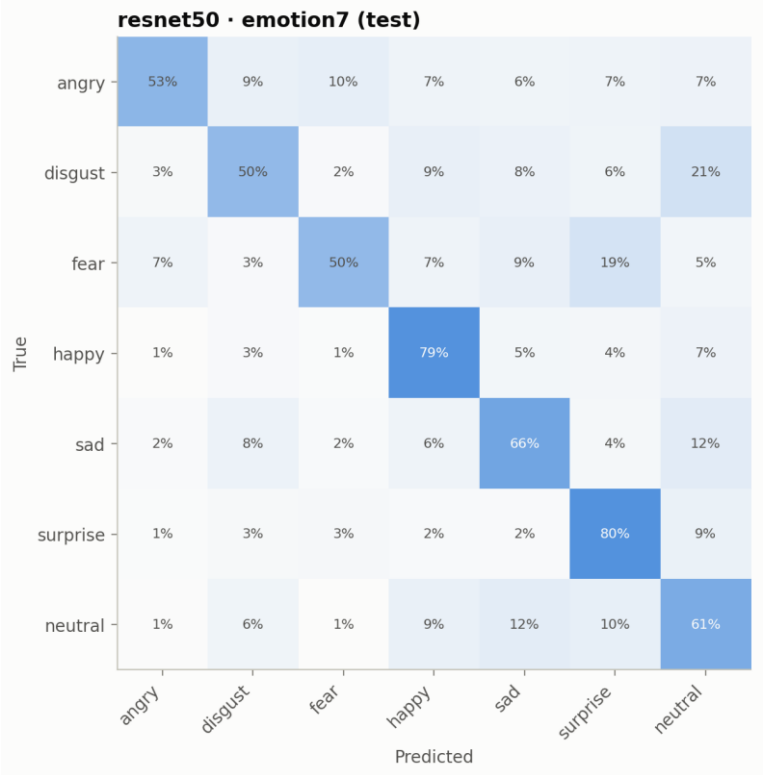


Figure 15. *Confusion matrix, RAF-DB, emotion7, ResNet50.*

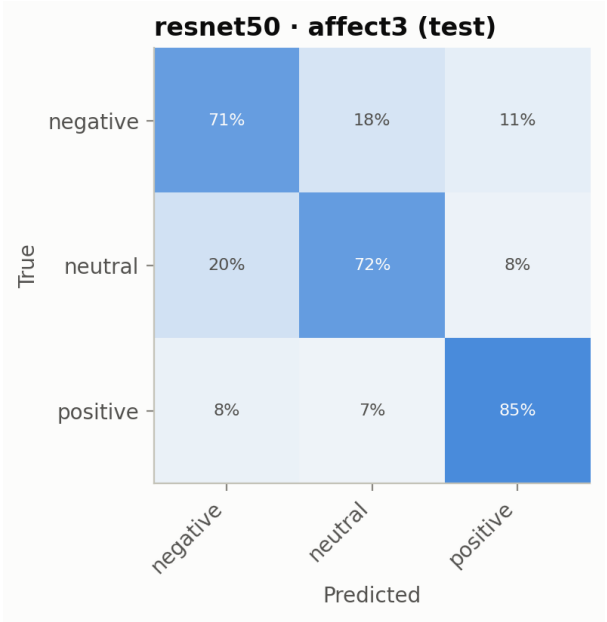


Figure 16. *Confusion matrix, FER-2013, affect3, ResNet50.*

The most common confusions are between *sad*, *neutral* and *fear*, and between *disgust* and *angry* or *neutral*. For the affect classes, most errors are between *negative* and *neutral*, while *positive* expressions are rarely mistaken for *negative* ones. This matches the intuition that subtle negative expressions are visually close to a neutral face.

4.6 Sample Predictions

Figures 17 and 18 show random correctly and incorrectly classified test images (true label → predicted label).

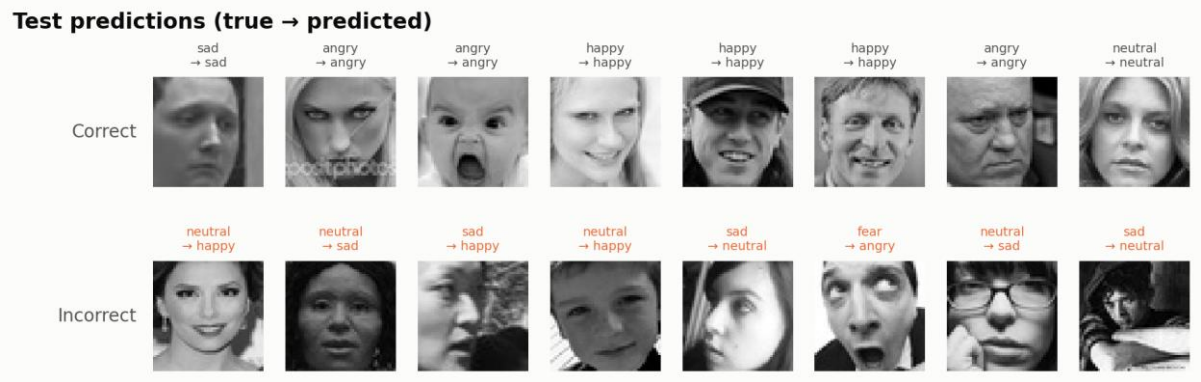


Figure 17. *Sample test predictions, FER-2013, emotion7, scratch CNN. Top: correct; bottom: incorrect.*



Figure 18. *Sample test predictions, RAF-DB, emotion7, ResNet50. Top: correct; bottom: incorrect.*

Many of the errors are understandable: several misclassified FER-2013 faces have ambiguous expressions, are partly occluded, are seen from the side or are very dark, and some labels themselves look questionable — a known property of FER-2013's single-annotator labels.

4.7 Efficiency

Table 7 also reports model size and speed. MobileNetV2 has the fewest parameters (2.3M), the scratch CNN 4.8M and ResNet50 23.6M. Nevertheless, the scratch CNN is by far the fastest at inference: 0.18 ms per image on FER-2013 and 0.65 ms on RAF-DB, against 1.15–1.31 ms for MobileNetV2 and 2.33–2.48 ms for ResNet50, because the pretrained models must process 224×224 inputs. On FER-2013 the pretrained models also took longer to train (28–45 minutes vs 15–27 minutes). On RAF-DB the scratch CNN took longer (32–74 minutes vs 14–19 minutes) because it trained for more epochs on its 100×100 colour inputs.

5. Output Screenshots from Google Colab

The complete pipeline was run in Google Colab on a GPU runtime. The figures in Sections 2 and 4 are the outputs produced by the notebook. The screenshots below show the notebook running.

[Insert Colab screenshot: runtime check showing the TensorFlow version and the GPU]

Figure 19. *Google Colab output: runtime check showing the TensorFlow version and the GPU.*

[Insert Colab screenshot: dataset preparation output with the train / validation / test sizes]

Figure 20. *Google Colab output: dataset preparation output with the train / validation / test sizes.*

[Insert Colab screenshot: training log of a transfer-learning model, showing phase 1 and phase 2 epochs]

Figure 21. *Google Colab output: training log of a transfer-learning model, showing phase 1 and phase 2 epochs.*

[Insert Colab screenshot: test-set evaluation output with accuracy and macro-F1 per run]

Figure 22. *Google Colab output: test-set evaluation output with accuracy and macro-F1 per run.*

[Insert Colab screenshot: comparison table produced by src.evaluation.compare]

Figure 23. *Google Colab output: comparison table produced by src.evaluation.compare.*

6. Analysis and Discussion

6.1 Did Transfer Learning Help?

Not in this setup. The CNN trained from scratch had the best macro-F1 in three of four settings, while also being the fastest at inference. On FER-2013 the differences are small (1–3 points) and, with a single training run per configuration, may be within run-to-run variation; on RAF-DB the scratch CNN is clearly better. The research question therefore has a nuanced answer: **for small, low-resolution face images, ImageNet pretraining with conservative fine-tuning did not provide an accuracy advantage over a well-regularised CNN trained directly on faces, and it was considerably slower at inference.**

6.2 Why Did the Pretrained Models Not Win?

- **Domain gap.** ImageNet contains photographs of objects, not close-up faces. The frozen features reached only 45–67% validation accuracy (Section 4.3), so most of the useful information had to be learned during fine-tuning.
- **Conservative fine-tuning.** Only the top stage of each backbone was unfrozen (38 of 154 layers for MobileNetV2, 32 of 175 for ResNet50), batch normalisation stayed frozen and the learning rate was 10^{-5}. This protects the pretrained features but limits adaptation. MobileNetV2 was still improving when training stopped at 60 epochs, so it was under-trained.
- **Overfitting of the large model.** ResNet50 fitted the training data quickly (training accuracy up to 0.92 on RAF-DB affect3) while validation accuracy stalled, which is typical of a 23.6M-parameter model on 11,000–26,000 images.
- **Upscaling.** FER-2013 images were upscaled from 48×48 to 224×224 and copied to three channels. This adds no information but forces the pretrained models to process 22 times more pixels, while the scratch CNN works at the native resolution.
- **Unequal tuning in one setting.** The RAF-DB emotion7 scratch CNN was trained with tuned settings (Section 3.6), while the pretrained models used defaults. This favours the scratch CNN in that setting and partly explains its large margin there; the other three comparisons used identical settings.

6.3 Accuracy versus Efficiency

ResNet50 has ten times more parameters than MobileNetV2 and was about twice as slow, but was only slightly more accurate (at most about 3 macro-F1 points, and slightly worse on FER-2013 emotion7). The scratch CNN had twice as many parameters as MobileNetV2 yet was the fastest model because it processes small images. For a practical system, for example one that analyses video in real time, the scratch CNN offers the best accuracy–speed trade-off in this study, and MobileNetV2 the smallest model.

6.4 Emotions versus Depression-Related Patterns

Grouping the emotions into negative, neutral and positive affect raised accuracy by 10–13 points on FER-2013 and by 4–10 points on RAF-DB, because many of the hardest confusions (for example *fear* versus *sad*) fall inside the *negative* group. The remaining errors are mostly between *negative* and *neutral*. For depression-related analysis this is the most important confusion, since flat affect and subtle negative expressions are both characteristic of depression and look similar on a single image.

6.5 Comparison with Published Results

On FER-2013, human accuracy is about 65% (Goodfellow et al., 2013), and carefully tuned single CNNs have reached about 73% (Khaireddin & Chen, 2021). Our best FER-2013 model (64.8%) is close to human level but below the state of the art, which is expected given that no extensive hyperparameter search, ensembles or test-time augmentation were used. RAF-DB results are higher for every model, consistent with its more reliable labels.

6.6 Ethical Considerations

The models recognise facial *expressions* associated with depression; they do **not** diagnose depression. Neither dataset contains clinical labels, facial expressions alone are not a valid basis for a mental-health diagnosis, and expressions depend on culture, context and individual differences. The datasets were collected from the internet and may not represent all ages, ethnicities and recording conditions equally, so performance may differ between groups. Any use of such models with real people would require informed consent, privacy protection and clinical oversight.

7. Limitations and Future Work

7.1 Limitations

- **Single run per configuration.** Each model was trained once (seed 42), so no standard deviations or significance tests can be given. The planned repetitions with three seeds and the experiment with 10–50% of the training data could not be completed in the available GPU time.
- **Unequal tuning.** One scratch CNN run used tuned settings, while the pretrained models used a single default configuration.
- **Partial fine-tuning only.** Only the top backbone stage was fine-tuned, with frozen batch normalisation and a fixed epoch limit.
- **Timing measurements.** Training and inference times were measured on shared Colab GPU runtimes and can vary between sessions.

- **No clinical validation.** The depression-related classes are derived from emotion labels, not from clinical data.

7.2 Future Work

- Repeat every configuration with several seeds and run the training-data-size experiment to test whether pretraining helps when less data is available.
- Give the pretrained models a fair tuning budget: unfreeze more layers or the whole backbone with layer-wise learning rates, fine-tune batch normalisation, and train for longer.
- Use input sizes closer to the native resolution (for example 96×96 or 128×128 for MobileNetV2) to reduce upscaling and speed up the pretrained models.
- Use backbones pretrained on faces (for example face-recognition models) instead of ImageNet, which should reduce the domain gap.
- Evaluate on video and on datasets with clinical depression labels, and analyse fairness across demographic groups.

8. Conclusion

This project compared a CNN trained from scratch with two ImageNet-pretrained models, MobileNetV2 and ResNet50, for classifying facial expressions into seven emotions and three depression-related affect patterns, on FER-2013 and RAF-DB. Under identical data, augmentation and evaluation, the scratch CNN achieved the best macro-F1 in three of four settings and was 2–13 times faster at inference. Fine-tuning was essential for the pretrained models, improving their validation accuracy by 10–19 points over frozen ImageNet features, but conservative fine-tuning of the top stage was not enough to close the gap to a network trained directly on faces. The best results were 64.8% accuracy for seven emotions on FER-2013 (scratch CNN), 78.7% on RAF-DB (scratch CNN), and 82.4% for the depression-related negative / neutral / positive classes on RAF-DB (scratch CNN). These results suggest that, for small face-expression datasets, a well-regularised task-specific CNN is a strong and efficient baseline, and that transfer learning needs a more thorough fine-tuning strategy, or face-specific pretraining, to pay off.

References

Abadi, M., Barham, P., Chen, J., Chen, Z., Davis, A., Dean, J., … Zheng, X. (2016). TensorFlow: A system for large-scale machine learning. *12th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, 265–283.

Chollet, F., et al. (2015). *Keras*. https://keras.io

Cohn, J. F., Kruez, T. S., Matthews, I., Yang, Y., Nguyen, M. H., Padilla, M. T., Zhou, F., & De la Torre, F. (2009). Detecting depression from facial actions and vocal prosody. *3rd International Conference on Affective Computing and Intelligent Interaction (ACII)*, 1–7.

Deng, J., Dong, W., Socher, R., Li, L.-J., Li, K., & Fei-Fei, L. (2009). ImageNet: A large-scale hierarchical image database. *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 248–255.

Girard, J. M., Cohn, J. F., Mahoor, M. H., Mavadati, S. M., Hammal, Z., & Rosenwald, D. P. (2014). Nonverbal social withdrawal in depression: Evidence from manual and automatic analyses. *Image and Vision Computing, 32*(10), 641–647.

Goodfellow, I. J., Erhan, D., Carrier, P. L., Courville, A., Mirza, M., Hamner, B., … Bengio, Y. (2013). Challenges in representation learning: A report on three machine learning contests. *International Conference on Neural Information Processing (ICONIP)*, 117–124.

He, K., Zhang, X., Ren, S., & Sun, J. (2016). Deep residual learning for image recognition. *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 770–778.

Ioffe, S., & Szegedy, C. (2015). Batch normalization: Accelerating deep network training by reducing internal covariate shift. *International Conference on Machine Learning (ICML)*, 448–456.

Khaireddin, Y., & Chen, Z. (2021). Facial emotion recognition: State of the art performance on FER2013. *arXiv preprint arXiv:2105.03588*.

Kingma, D. P., & Ba, J. (2015). Adam: A method for stochastic optimization. *International Conference on Learning Representations (ICLR)*.

Li, S., & Deng, W. (2022). Deep facial expression recognition: A survey. *IEEE Transactions on Affective Computing, 13*(3), 1195–1215.

Li, S., Deng, W., & Du, J. (2017). Reliable crowdsourcing and deep locality-preserving learning for expression recognition in the wild. *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2852–2861.

Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., … Duchesnay, É. (2011). Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research, 12*, 2825–2830.

Sandler, M., Howard, A., Zhu, M., Zhmoginov, A., & Chen, L.-C. (2018). MobileNetV2: Inverted residuals and linear bottlenecks. *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 4510–4520.

Srivastava, N., Hinton, G., Krizhevsky, A., Sutskever, I., & Salakhutdinov, R. (2014). Dropout: A simple way to prevent neural networks from overfitting. *Journal of Machine Learning Research, 15*, 1929–1958.

Yosinski, J., Clune, J., Bengio, Y., & Lipson, H. (2014). How transferable are features in deep neural networks? *Advances in Neural Information Processing Systems (NeurIPS), 27*, 3320–3328.

FER-2013 dataset (Kaggle): https://www.kaggle.com/datasets/msambare/fer2013

RAF-DB dataset (Kaggle copy): https://www.kaggle.com/datasets/shuvoalok/raf-db-dataset; official: http://www.whdeng.cn/RAF/model1.html

Appendix A. Reproducing the Results

The code is available at https://github.com/SaraBedulla/FacemoodClassifier. The Colab notebook (open in Colab) clones the repository, downloads the data from Kaggle and runs every step. Locally, the same steps are:

```
python -m src.data.prepare --dataset fer2013 --source data/raw/fer2013
python -m src.data.prepare --dataset rafdb --source data/raw/rafdb
python -m src.training.train --dataset fer2013 --model scratch_cnn --labels emotion7
python -m src.training.train --dataset fer2013 --model mobilenetv2 --labels emotion7
python -m src.training.train --dataset fer2013 --model resnet50 --labels emotion7
python -m src.evaluation.evaluate
python -m src.evaluation.compare
```

The same training command is repeated for each dataset (`--dataset fer2013` or `rafdb`), model and label scheme (`--labels emotion7` or `affect3`). The RAF-DB emotion7 scratch CNN additionally used `--lr 0.0003 --epochs 100 --patience 20`.

Appendix B. Training Curves and Confusion Matrices of the Remaining Models

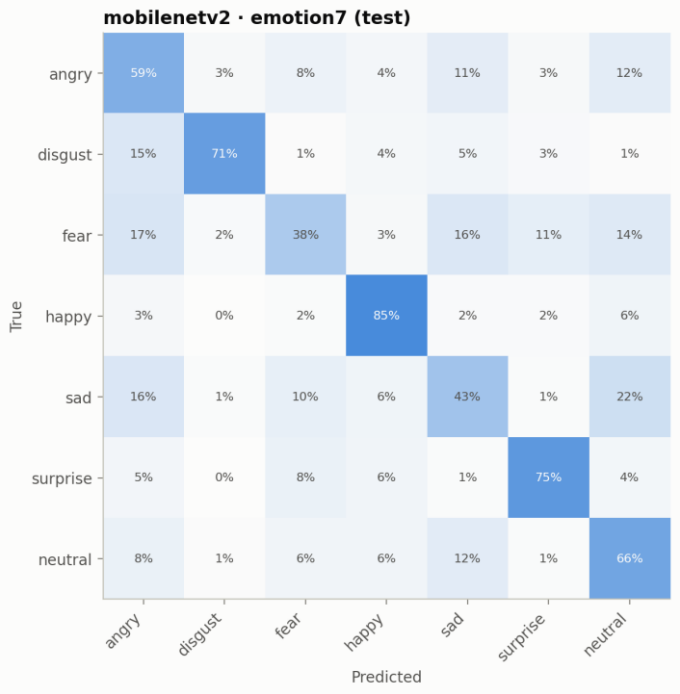


Figure 24. *FER-2013, emotion7, MobileNetV2: confusion matrix (test).*

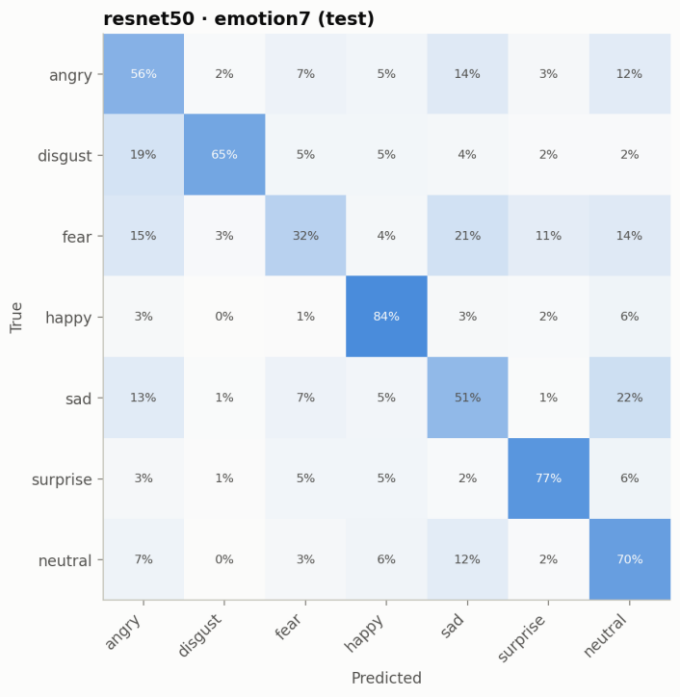


Figure 25. *FER-2013, emotion7, ResNet50: confusion matrix (test).*

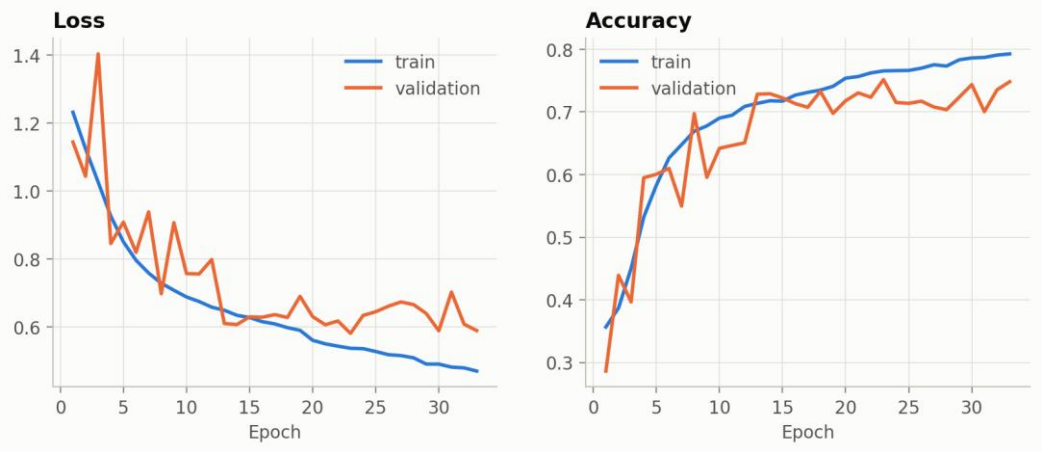


Figure 26. *FER-2013, affect3, Scratch CNN: training and validation curves.*

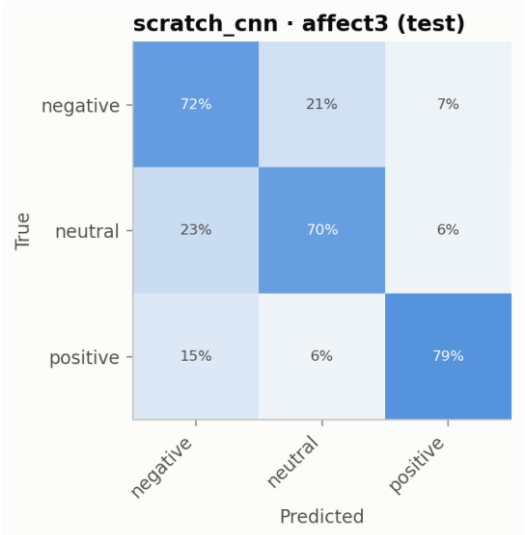


Figure 27. *FER-2013, affect3, Scratch CNN: confusion matrix (test).*

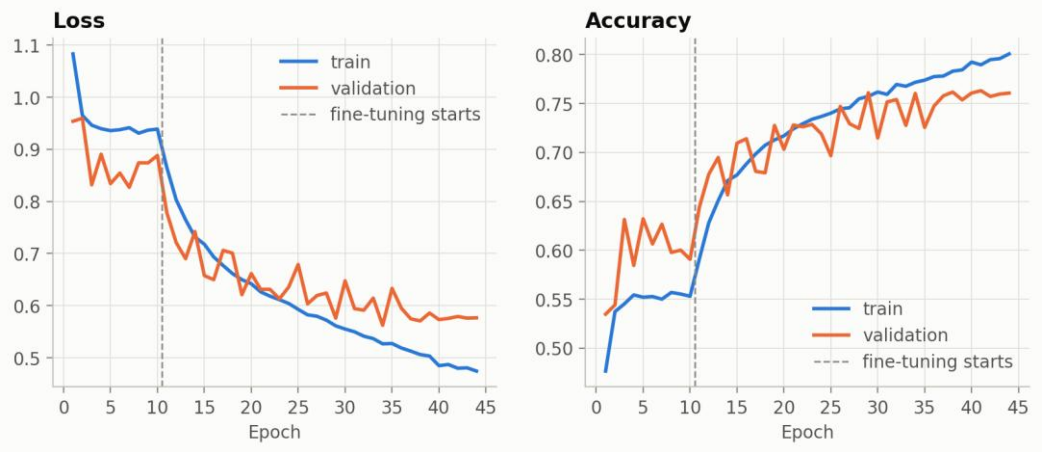


Figure 28. *FER-2013, affect3, MobileNetV2: training and validation curves.*

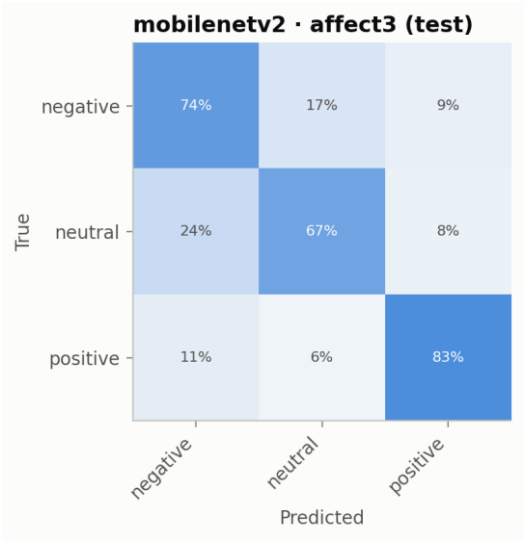


Figure 29. *FER-2013, affect3, MobileNetV2: confusion matrix (test).*

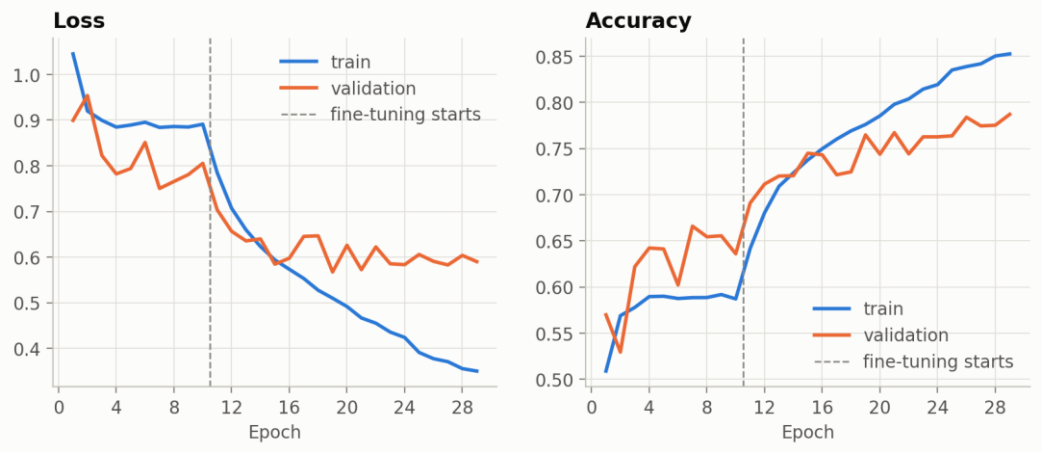


Figure 30. *FER-2013, affect3, ResNet50: training and validation curves.*

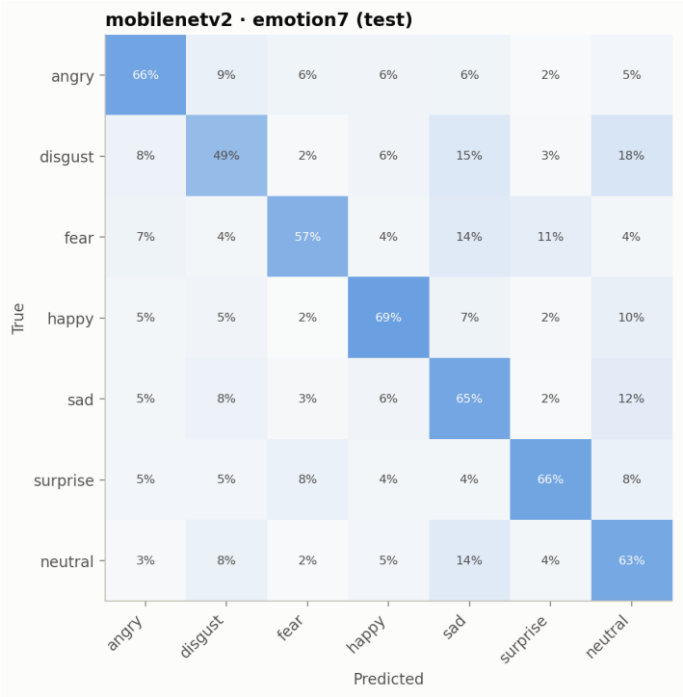


Figure 31. *RAF-DB, emotion7, MobileNetV2: confusion matrix (test).*

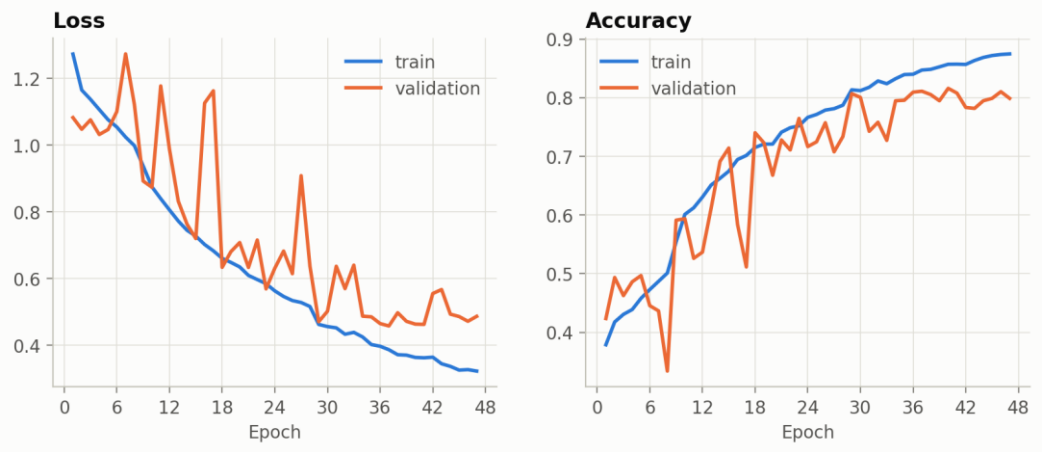


Figure 32. *RAF-DB, affect3, Scratch CNN: training and validation curves.*

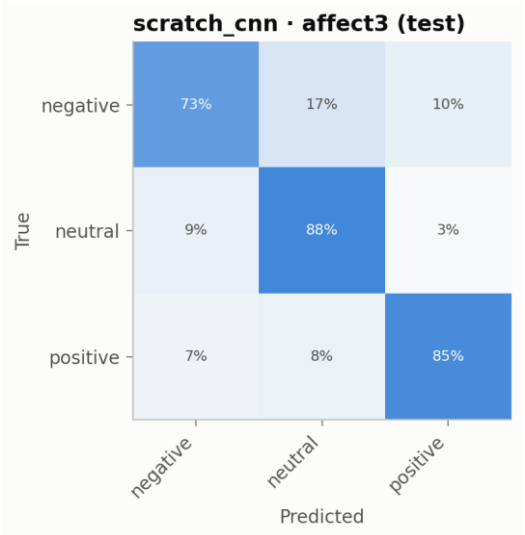


Figure 33. *RAF-DB, affect3, Scratch CNN: confusion matrix (test).*

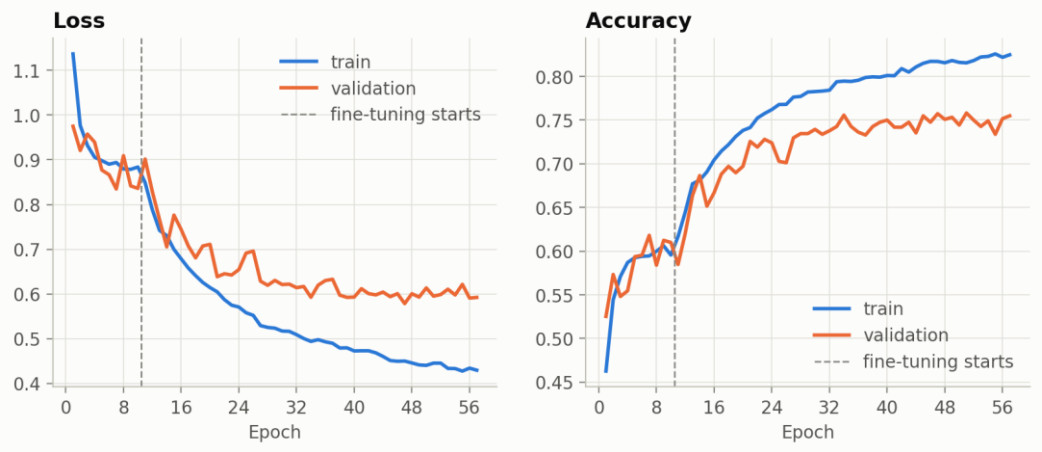


Figure 34. *RAF-DB, affect3, MobileNetV2: training and validation curves.*

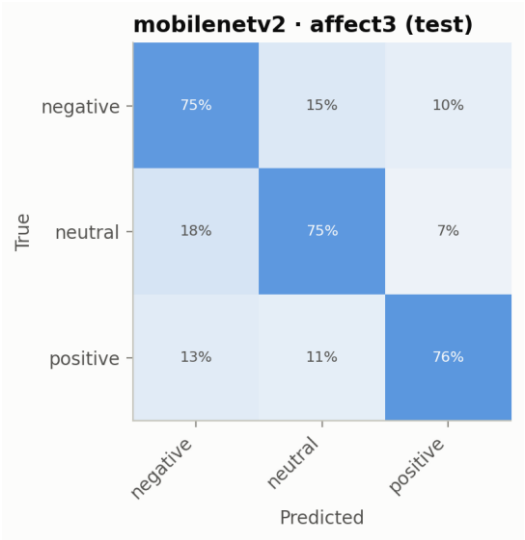


Figure 35. *RAF-DB, affect3, MobileNetV2: confusion matrix (test).*

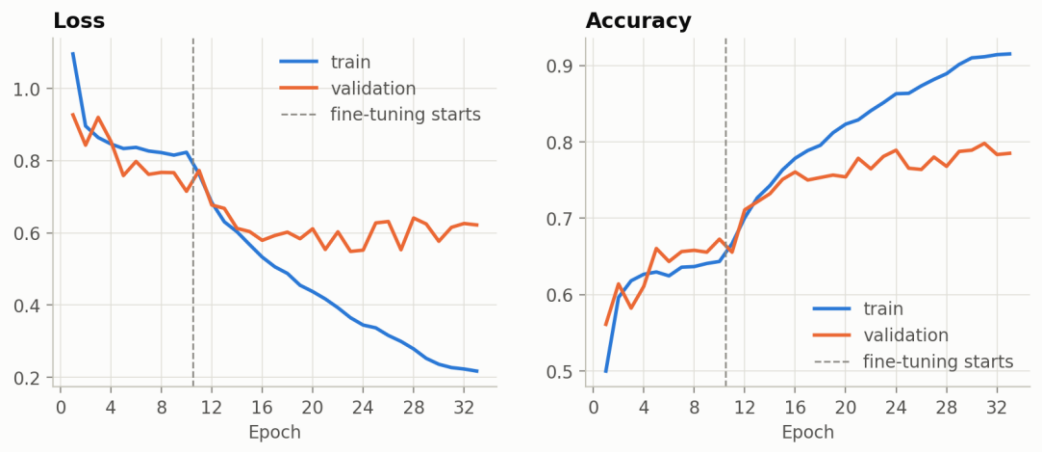


Figure 36. *RAF-DB, affect3, ResNet50: training and validation curves.*

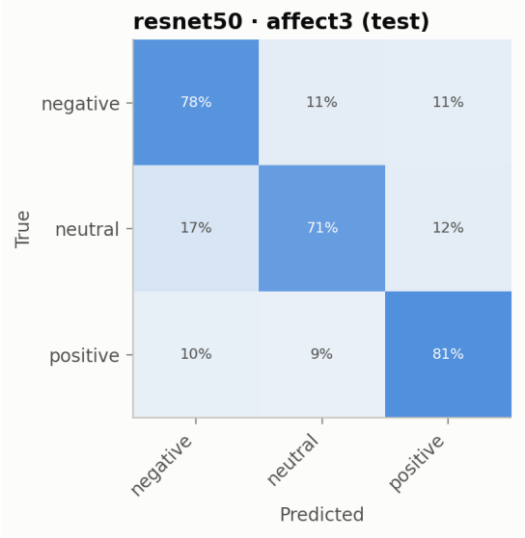


Figure 37. *RAF-DB, affect3, ResNet50: confusion matrix (test).*